

Binary Number System

► **Fundamental Principle:**

- Base-2 positional system
- Only two digits: 0 and 1
- Each position represents power of 2

► **Example:** Binary 1101 to Decimal

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \quad (4)$$

$$= 8 + 4 + 0 + 1 = 13_{10} \quad (5)$$

Binary Addition Example

Binary Addition Rules:

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10_2 \text{ (carry 1)}$$

(6)

(7)

(8)

(9)

Example: $1011_2 + 1101_2$

$$\begin{array}{r} 1011 \\ + 1101 \\ \hline 11000 \end{array}$$

Verification: $11 + 13 = 24 = 11000_2 \checkmark$

Two's Complement System

- ▶ **Signed Binary Representation:**

- ▶ Standard for representing negative numbers
- ▶ Simplifies arithmetic operations
- ▶ No separate subtraction circuitry needed

- ▶ **Two's Complement Process:**

1. Invert all bits (one's complement)
2. Add 1 to the result

- ▶ **Example:** -5 in 4-bit two's complement

- ▶ +5: 0101
- ▶ Invert: 1010
- ▶ Add 1: 1011 = -5

Two's Complement Range

► n-bit Two's Complement Range:

- Range: -2^{n-1} to $2^{n-1} - 1$
- Most significant bit is sign bit
- Asymmetric range (one more negative than positive)

Bits	Range	Example
4	-8 to +7	1000 = -8, 0111 = +7
8	-128 to +127	10000000 = -128
16	-32,768 to +32,767	Standard short int
32	-2^{31} to $2^{31} - 1$	Standard int

Excess-k (Biased) Representation

- ▶ **Biased Number System:**
 - ▶ Add a bias (k) to the actual value
 - ▶ Useful for floating-point exponents
 - ▶ Simplifies comparison operations
- ▶ **Example:** Excess-127 (8-bit)
 - ▶ Bias = 127
 - ▶ Range: -127 to +128
 - ▶ Stored value = Actual value + 127
 - ▶ Zero is represented as 127

Actual Value	Stored Value	Binary
-127	0	00000000
-1	126	01111110
0	127	01111111
+1	128	10000000
+128	255	11111111

► **Practical Solution:**

- For embedded systems and early computers
- Limited hardware complexity
- Predictable precision

► **Format:** *s.f* (sign.fractional) decimal point position

- Integer and fractional parts
- Example: 8.8 format (8 integer, 8 fractional bits)

Advantages:

- Simple arithmetic operations
- Predictable range and precision
- Lower hardware cost

Fixed-Point Example

Example: 16-bit fixed-point with 8.8 format

- ▶ **Representation:** I.IIIIIII.FFFFFFFF
- ▶ **Range:** -128.0 to +127.99609375
- ▶ **Resolution:** $2^{-8} = 0.00390625$

Examples:

Decimal	Binary (8.8)	Hex
12.25	00001100.01000000	0C40
-5.5	11111010.10000000	FA80
0.75	00000000.11000000	00C0

Dynamic Range Solution

► **Motivation:**

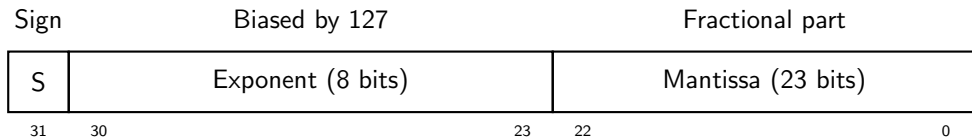
- Need for wide dynamic range
- Scientific and engineering applications
- Automatic scaling of numbers

► **IEEE 754 Standard:**

- Sign bit (1 bit)
- Exponent (8 bits for single precision)
- Mantissa/Significand (23 bits for single precision)

► **Format:** $(-1)^s \times 1.M \times 2^{E-127}$

IEEE 754 Single Precision Format



Key Properties:

- ▶ **Range:** $\pm 1.18 \times 10^{-38}$ to $\pm 3.40 \times 10^{38}$
- ▶ **Precision:** About 7 decimal digits
- ▶ **Special values:** ± 0 , $\pm \infty$, NaN

Floating-Point Example

Example: Represent 12.375 in IEEE 754 single precision

1. Convert to binary: $12.375 = 1100.011_2$
2. Normalize: 1.100011×2^3
3. Sign bit: 0 (positive)
4. Exponent: $3 + 127 = 130 = 10000010_2$
5. Mantissa: 1000110000000000000000_2 (23 bits)

Result:

S	Exponent	Mantissa
0	10000010	10001100000000000000000

Verification: $(-1)^0 \times 1.100011_2 \times 2^{130-127} = 1.100011_2 \times 2^3 = 12.375$

BCD Representation

► Hybrid System:

- Represents decimal digits in binary
- Each decimal digit uses 4 bits
- Maintains decimal semantics

► Example: 1234 in BCD

$$1234_{10} = 0001\ 0010\ 0011\ 0100_{BCD} \quad (10)$$

$$\neq 10011010010_2 \text{ (pure binary)} \quad (11)$$

► Applications:

- Financial calculations (no rounding errors)
- Display systems (7-segment displays)

BCD vs. Binary Comparison

Decimal	BCD	Pure Binary
0	0000	0000
1	0001	0001
2	0010	0010
9	1001	1001
10	0001 0000	1010
15	0001 0101	1111
99	1001 1001	1100011
255	0010 0101 0101	11111111

Trade-offs:

- ▶ **BCD**: Exact decimal representation, easier decimal I/O
- ▶ **Binary**: More compact, faster arithmetic

Key Takeaways

▶ **Historical Evolution:**

- ▶ From tally marks to sophisticated digital systems
- ▶ Each system solved specific computational needs
- ▶ Positional notation was revolutionary

▶ **Mathematical Foundations:**

- ▶ Number sets: $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \mathbb{C}$
- ▶ Positional notation principles
- ▶ Complement arithmetic for subtraction

▶ **Digital Computing Requirements:**

- ▶ Binary representation for electronic systems
- ▶ Multiple formats for different applications
- ▶ Trade-offs between range, precision, and complexity

Modern Applications

▶ **Computer Architecture Impact:**

- ▶ All digital systems use these principles
- ▶ Hardware design based on number representations
- ▶ Software algorithms optimized for binary operations

▶ **Representation Trade-offs:**

- ▶ **Fixed-point:** Simple, predictable, limited range
- ▶ **Floating-point:** Wide range, complex, IEEE standard
- ▶ **BCD:** Exact decimal, larger storage requirement
- ▶ **Two's complement:** Efficient signed arithmetic

▶ **Future Considerations:**

- ▶ Quantum computing representations
- ▶ Approximate computing systems

Questions and Discussion

Questions?

Discussion Topics:

- ▶ Why did binary become the standard for digital computers?
- ▶ What are the advantages of two's complement over other signed representations?
- ▶ When would you choose fixed-point over floating-point arithmetic?
- ▶ How do number representation choices affect computer architecture?

Next: Chapter 2 - Formal Notation and Mathematical Foundations